

Manual tecnico dos scripts operacionais

Este manual descreve os scripts em `scripts/ops` para operar os ambientes `live` e `sustenido`.

Modelo de ambientes

- `live`: ambiente de desenvolvimento/staging.
- `sustenido`: ambiente de producao.
- Banco de dados: ambientes separados, mas com os mesmos nomes de databases Mongo o por padrao: `liveNlound` e `generalCifras`.
- Frontend: publicado como arquivos estaticos em diretorios diferentes no VPS.
- Backend: atualizado via `git pull --ff-only` e `docker compose up -d --build`.

Arquivos base

`config.example.sh`

Modelo de configuracao. Deve ser copiado para:

```
```bash
scripts/ops/config.sh
```

O `config.sh` fica fora do Git e contem valores reais como:

- `REMOTE\_SERVER`: usuario e host SSH.
- `LOCAL\_BACKUP\_DIR`: destino local dos backups baixados do VPS.
- `LIVE\_FRONT\_SITE\_DIR` e `SUSTENIDO\_FRONT\_SITE\_DIR`: diretorios estaticos do front.
- `LIVE\_REMOTE\_APP\_DIR` e `SUSTENIDO\_REMOTE\_APP\_DIR`: diretorios remotos que contem `docker-compose.yml`.
- `MONGO\_USER` e `MONGO\_PASS`: credenciais Mongo.
- `MONGO\_AUTH\_DATABASE`: database de autenticao, normalmente `admin`.
- `MONGO\_ROOT\_ROLES`: roles usadas ao criar usuario admin em Mongo novo.
- `MONGO\_DATABASES`: lista de databases copiadas/backupeadas.
- `BACKEND\_SERVICES`: servicos recreados no deploy backend.

#### ### `lib.sh`

Biblioteca compartilhada pelos scripts. Ela:

- carrega `config.sh`;
- valida comandos locais (`ssh`, `scp`, `rsync`, etc.);
- resolve configuracao por ambiente;
- executa confirmacoes fortes;
- executa `mongodump`, `mongorestore`, `scp`, `rsync`;
- executa `docker compose` remoto;
- executa deploy de `Front/dist`;
- executa build completo do front;
- executa healthchecks.

### ## Configuracao inicial

No computador local:

```
```bash
cp scripts/ops/config.example.sh scripts/ops/config.sh
```

Edite `scripts/ops/config.sh`. Para a estrutura mostrada no VPS, valores provaveis:

```
```bash
LIVE_REMOTE_APP_DIR="/home/liveNloud"
SUSTENIDO_REMOTE_APP_DIR="/home/sustenido"
```
```

Esses diretorios precisam conter o arquivo definido por `COMPOSE\_FILE`, normalmente:

```
```bash
docker-compose.yml
```
```

Backups de banco

Backup manual de live

```
```bash
./scripts/ops/db/bkp/backup_live_db.sh
```
```

Fluxo tecnico:

1. Carrega `config.sh`.
2. Verifica se `LIVE\_REMOTE\_APP\_DIR/\$COMPOSE\_FILE` existe no VPS.
3. Para cada database em `MONGO\_DATABASES`, executa `mongodump` dentro do servico Mongo.
4. Salva o archive no VPS em:

```
```bash
$LIVE_REMOTE_APP_DIR/backups/mongo/<timestamp>-manual/
```
```

5. Baixa uma copia local para:

```
```bash
$LOCAL_BACKUP_DIR/live/<timestamp>-manual/
```
```

Backup manual de sustenido

```
```bash
./scripts/ops/db/bkp/backup_sustenido_db.sh
```
```

Mesmo fluxo do live, mas usando `SUSTENIDO\_REMOTE\_APP\_DIR` e salvando em:

```
```bash
$LOCAL_BACKUP_DIR/sustenido/<timestamp>-manual/
```
```

Escopo do backup

Por padrao, os scripts fazem backup apenas dos databases listados em:

```
```bash
MONGO_DATABASES=("liveNloud_" "generalCifras")
```
```

Isso inclui todas as collections e indices dentro desses databases da aplicacao.

Os scripts nao fazem dump dos databases internos:

```
```text
admin
config
local
```
```

Motivo:

- `admin`: contem usuarios e roles Mongo.
- `config`: contem metadados internos.
- `local`: contem dados internos do servidor Mongo, como `startup\_log` e dados de replicação.

Para restaurar a aplicação em um servidor novo, o usuário Mongo deve ser recriado ou por configuração (`.env`/Docker) ou pelo script de bootstrap descrito abaixo. Depois disso, os arquivos de `liveNlcloud` e `generalCifras` restauram os dados da aplicação.

Bootstrap de Mongo novo

Use quando o volume Mongo está vazio, quando o servidor foi recriado do zero ou quando o usuário admin ainda não existe.

Live:

```
```bash
./scripts/ops/db/bootstrap/bootstrap_mongo_live.sh
```
```

Sustenido:

```
```bash
./scripts/ops/db/bootstrap/bootstrap_mongo_sustenido.sh
```
```

Frases de confirmação:

```
```text
BOOTSTRAP MONGO LIVE
BOOTSTRAP MONGO SUSTENIDO
```
```

Fluxo técnico:

1. Valida que o diretório remoto contém `COMPOSE\_FILE`.
2. Executa:

```
```bash
docker compose -f "$COMPOSE_FILE" up -d "$MONGO_SERVICE"
```
```

3. Aguarda o Mongo responder `ping`.
4. Tenta autenticar usando:

```
```bash
MONGO_USER
MONGO_PASS
MONGO_AUTH_DATABASE
```
```

5. Se a autenticação já funciona, não altera nada.

6. Se a autenticação falha, tenta conectar sem credenciais e criar o usuário:

```
```javascript
db.getSiblingDB(MONGO_AUTH_DATABASE).createUser({
 user: MONGO_USER,
 pwd: MONGO_PASS,
 roles: MONGO_ROOT_ROLES
})
```
```

7. Revalida a autenticação.

Limite importante: se o usuário já existir e a senha estiver errada, o script falha. Nesse caso, corrija `MONGO\_PASS` em `config.sh` ou ajuste manualmente o usuário no Mongo.

Sincronização de bancos

Live para sustenido

```
```bash
./scripts/ops/db/sync_db/db_live_to_sustenido.sh
```
```

Operação destrutiva para o destino. O script exige a frase:

```
```text
COPY LIVE TO SUSTENIDO
```
```

Fluxo técnico:

1. Faz backup do destino `sustenido`.
2. Faz backup da origem `live`.
3. Usa o backup local da origem como fonte.
4. Envia os archives para o VPS de destino.
5. Executa `mongorestore --drop` para cada database em `MONGO\_DATABASES`.
6. Executa healthcheck de sustenido.

Sustenido para live

```
```bash
./scripts/ops/db/sync_db/db_sustenido_to_live.sh
```
```

Operação destrutiva para o destino. O script exige a frase:

```
```text
COPY SUSTENIDO TO LIVE
```
```

Fluxo técnico equivalente ao anterior, invertendo origem e destino.

Restauração de bancos a partir de backups

Os backups locais ficam no formato:

```
```bash
$LOCAL_BACKUP_DIR/<ambiente>/<timestamp>-<label>/
```
```

Dentro da pasta devem existir archives como:

```
```bash
liveNloud_.archive.gz
generalCifras.archive.gz
```

### Restaurar backup para live

```
```bash
./scripts/ops/db/restore/restore_live_db_from_backup.sh "$HOME/LiveNloud-db-backups/live/20260605-110000-manual"
```

O script exige a frase:

```
```text
RESTORE BACKUP TO LIVE
```

Fluxo tecnico:

1. Executa bootstrap/verificacao do usuario admin Mongo.
2. Faz backup preventivo de live antes de sobrescrever.
3. Envia os archives locais para:

```
```bash
$LIVE_REMOTE_APP_DIR/backups/incoming-restore/<timestamp>/
```

4. Copia cada archive para dentro do container Mongo.
5. Executa `mongorestore --drop` por database.
6. Remove o archive temporario dentro do container.
7. Executa healthcheck de live.

Restaurar backup para sustenido

```
```bash
./scripts/ops/db/restore/restore_sustenido_db_from_backup.sh "$HOME/LiveNloud-db-backups/sustenido/20260605-110000-manual"
```

O script exige a frase:

```
```text
RESTORE BACKUP TO SUSTENIDO
```

Fluxo tecnico equivalente ao restore de live, usando `SUSTENIDO\_REMOTE\_APP\_DIR`.

Restauracao em servidor novo do zero

Este procedimento cobre o caso em que voce perdeu o volume Mongo ou esta subindo um VPS novo.

Pre-requisitos no novo servidor

1. Codigo/backend presente no directorio remoto configurado:

```
```bash
LIVE_REMOTE_APP_DIR
SUSTENIDO_REMOTE_APP_DIR
```

2. `docker-compose.yml` presente no diretorio remoto.
3. `.env` do ambiente recriado com os mesmos valores essenciais:

```
```bash
MONGO_URI
PYTHON_API_URL
ACCESS_SECRET
REFRESH_SECRET
FRONTEND_BASE_URL
API_PUBLIC_BASE_URL
SMTP_*
```
```

4. `config.sh` local apontando para o novo servidor/diretorios e com:

```
```bash
MONGO_USER
MONGO_PASS
MONGO_AUTH_DATABASE="admin"
MONGO_DATABASES=("liveNloud_" "generalCifras")
```
```

### ### Passo a passo

1. Suba/crie o usuario Mongo:

```
```bash
./scripts/ops/db/bootstrap/bootstrap_mongo_live.sh
```
```

ou:

```
```bash
./scripts/ops/db/bootstrap/bootstrap_mongo_sustenido.sh
```
```

2. Restaure os archives locais:

```
```bash
./scripts/ops/db/restore/restore_live_db_from_backup.sh "$HOME/LiveNloud-db-backups/live/PASTA_DO_BACKUP"
```
```

ou:

```
```bash
./scripts/ops/db/restore/restore_sustenido_db_from_backup.sh "$HOME/LiveNloud-db-backups/sustenido/PASTA_DO_BACKUP"
```
```

3. Suba o backend:

```
```bash
./scripts/ops/deploy/back/deploy_backend_live.sh
```
```

ou:

```
```bash
./scripts/ops/deploy/back/deploy_backend_sustenido.sh
```
```

4. Rode um backup de verificacao:

```
```bash
./scripts/ops/db/bkp/backup_live_db.sh
```

ou:

```
```bash
./scripts/ops/db/bkp/backup_sustenido_db.sh
```

5. Confira no MongoDB Compass se `liveNlound` e `generalCifras` possuem as collections esperadas.

### 0 que este processo nao restaura

- Databases internos `admin`, `config`, `local`.
- Usuarios Mongo alem do usuario admin criado pelo bootstrap.
- Arquivos fora do Mongo, como uploads em disco, se existirem.
- Secrets de `.env`.

Esses itens devem ser recriados via Docker Compose, `.env` ou backup separado de arquivos.

## Deploy completo de front

Estes sao os scripts corretos para deploy completo no fluxo normal.

### Live

```
```bash
./scripts/ops/deploy/front/deploy_front_live.sh
```

Substitui a logica do antigo:

```
```bash
Front/deploy/deploy_livenlound.sh
```

Fluxo tecnico:

1. Busca `GIT\_REMOTE/GIT\_BRANCH`.
2. Le a versao em `Front/\$SOFT\_VERSION\_FILE` no `FETCH\_HEAD`.
3. Incrementa o ultimo segmento da versao.
4. Atualiza `SoftVersion.jsx` e `SoftVersion.test.jsx`.
5. Executa `npm test`.
6. Executa `npm run build:live`.
7. Valida se `Front/dist` contem `LIVE\_EXPECTED\_API\_BASE`.
8. Valida se `Front/dist` nao contem `LIVE\_FORBIDDEN\_API\_BASE`.
9. Faz backup do site remoto.
10. Publica `Front/dist` com `rsync --delete`.
11. Executa healthcheck de live.

### Sustenido

```
```bash
./scripts/ops/deploy/front/deploy_front_sustenido.sh
```

Substitui a logica do antigo:

```
```bash
Front/deploy/deploy_sustenido.sh
```
```

Fluxo tecnico equivalente ao live, mas usando:

```
```bash
npm run build:sustenido
SUSTENIDO_EXPECTED_API_BASE
SUSTENIDO_FORBIDDEN_API_BASE
```
```

Deploy de `Front/dist` ja pronto

Voce so usaria `deploy\_dist\_\*` em casos especificos, por exemplo:

- voce ja rodou o build manualmente;
- o teste/build ja passou e voce so quer reenviar o mesmo `dist`;
- deu erro de rede no `rsync` e voce quer repetir so o envio;
- quer publicar um `dist` gerado por outro processo.

Se voce nao tem esse caso, pode ignorar os `deploy\_dist\_\*`.

Live:

```
```bash
cd Front
npm run build:live
cd ..
./scripts/ops/deploy/front/manual_dist/deploy_dist_live.sh
```
```

Sustenido:

```
```bash
cd Front
npm run build:sustenido
cd ..
./scripts/ops/deploy/front/manual_dist/deploy_dist_sustenido.sh
```
```

Esses scripts nao incrementam versao e nao rodam testes. Eles apenas validam API no `dist`, fazem backup remoto e publicam com `rsync`.

Deploy backend

Live

```
```bash
./scripts/ops/deploy/back/deploy_backend_live.sh
```
```

Sustenido

```
```bash
./scripts/ops/deploy/back/deploy_backend_sustenido.sh
```
```

Fluxo tecnico:

1. Valida se o diretorio remoto contem `COMPOSE\_FILE`.
2. Executa `git pull --ff-only` no diretorio remoto.
3. Executa:

```
```bash
docker compose -f "$COMPOSE_FILE" up -d --build node python_scraper
```

4. Executa healthcheck do ambiente.

## Ordem recomendada de uso

Para um deploy normal de backend:

```
```bash
./scripts/ops/deploy/back/deploy_backend_live.sh
./scripts/ops/deploy/back/deploy_backend_sustenido.sh
```

Para publicar front em live:

```
```bash
./scripts/ops/deploy/front/deploy_front_live.sh
```

Para publicar front em producao:

```
```bash
./scripts/ops/deploy/front/deploy_front_sustenido.sh
```

Para copiar dados de producao para dev:

```
```bash
./scripts/ops/db/sync_db/db_sustenido_to_live.sh
```

Para promover dados de dev para producao, use somente quando tiver certeza:

```
```bash
./scripts/ops/db/sync_db/db_live_to_sustenido.sh
```

Riscos e controles

- Scripts de banco usam frases exatas de confirmacao.
- Todo overwrite de banco faz backup antes.
- Backups ficam no VPS e tambem no computador local.
- Deploy de front valida API esperada e API proibida.
- Deploy de backend usa `git pull --ff-only`; se houver divergencia no VPS, o de ploy falha em vez de fazer merge automatico.